Welcome to this comprehensive guide on creating an AI-powered study buddy application using the Crew AI framework. In this course, you will learn how to build an application that reads PDF documents, generates concise summaries, and tests your knowledge through quizzes and flashcards. This application will utilize multiple AI agents working together seamlessly to enhance your learning experience.

Prerequisites

Before diving into the course, ensure you meet the following prerequisites:

- **Python Proficiency:** You should be comfortable with the Python programming language.
- **Python Development Tools:** Ensure that Python development tools are installed on your machine.
- **Command Line Interface (CLI) Knowledge:**
  - Windows users should be familiar with PowerShell.
  - Unix or Linux users should be familiar with Bash.
- **Crew AI Framework Knowledge:** A basic understanding of the Crew AI framework is essential. If you are new to Crew AI, consider taking relevant courses on Zenva Academy.
- **Crew AI and Tools Installation:** Ensure that Crew AI and its associated tools are installed on your development machine.

Learning Goals

Throughout this course, you will achieve the following learning goals:

1. Set up an agent-index application using the Crew AI framework.
2. Learn how to manage agents and tasks within the framework.
3. Build agents capable of processing human input using OpenAI's API.
4. Integrate local large language models into your Crew AI project.
5. Create a flashcard mechanism in the terminal with commands like show, flip, and next.

Project Structure

The project will cover the following topics:

- **Folder Structure:** Understand the project's folder structure, including agent definitions, tasks, and the main Crew file.
- **Content Summarization:** Learn how content is summarized and how quizzes are generated automatically.
- **Reading and Summarizing Data:** Extract and summarize text content from PDF files.
- **Quiz and Flashcard Generation:** Implement algorithms to handle human input and agent responses for quiz and flashcard generation.

About Zenva Academy

Zenva Academy is an online learning platform with over a million students. It offers a wide range of courses suitable for beginners and those looking to learn new skills. You can choose to watch video tutorials, read lesson summaries, or follow along with the included project files.

Now that you have an overview of what to expect, let's get started on building your AI-powered study buddy application!

Before diving into the project, it's essential to have a clear plan and preparation. This project involves setting up your development environment, creating a virtual environment, and defining agents and tasks. The ultimate goal is to build an application where users can interact with various agents to read PDF documents, summarize content, generate quizzes, and create flashcards.

Project Setup

To begin, ensure you have the following prerequisites:

- **Python Version:** Confirm that you have Python 3.10 or later installed on your machine.
- **CrewAI and CrewAI Tools:** Install CrewAI and CrewAI tools in your development environment. If you're unsure how to install them, refer to other courses available on Zenva Academy or follow the brief instructions in the upcoming sessions.

Creating a Virtual Environment

A virtual environment helps keep your project self-contained. Here's how to set it up:

1. **Folder Structure:** Organize your project with a clear folder structure. Place your code in the source folder and configurations in the config folder.
2. **API Access Keys:** Manage secret API access keys using a .env file.
3. **Version Control (Optional):** You can use Git for version control, but it's optional for this tutorial series.

Defining Agents and Tasks

After setting up your project, the next step is to define agents and their tasks. Identify the roles of each agent:

- **PDF Reader Agent:** Reads PDF documents.
- **Summarizer Agent:** Summarizes the content of the PDFs.
- **Quiz Maker Agent:** Generates quizzes based on the summarized content.
- **Flashcard Generator Agent:** Creates flashcards for interactive learning.

To ensure these agents perform well with human inputs, define each agent's purpose and backstory. The task sequence is as follows:

1. Reading PDF documents.
2. Summarizing the content.
3. Generating quizzes.
4. Interacting with the generated flashcards.

User Interaction

For the application to be functional, users must be able to interact with these agents by typing in the terminal. This requires a small feedback mechanism, which is one of the more challenging but interesting parts of the project. Don't worry; guidance will be provided throughout the process.

Defining Agents and Tasks in YAML Files

All agents and tasks will be defined in the agent and tasks.yaml files. This ensures that your project is well-organized and easy to manage.

By following these steps, you'll be well on your way to building a robust application using CrewAI and CrewAI tools. Stay tuned for more detailed instructions in the upcoming sessions.

In this session, we will set up our CrewAI project. This involves creating a virtual environment, installing necessary packages, and initializing our project with specific configurations. Let’s go through the steps one by one.

Creating a Virtual Environment

First, we need to create a virtual environment to isolate our project dependencies. This ensures that our project’s packages do not interfere with other projects or the system’s Python installation.

Open your terminal or command prompt.
Navigate to your project directory. In the video, the directory is named StudyBuddy, but you can name it anything you like:

```
cd path/to/your/project/directory
```

Create a virtual environment named venv by running:

```
python -m venv venv
```

Activating the Virtual Environment

After creating the virtual environment, we need to activate it:

On Windows:

```
venv\Scripts\activate
```

On macOS and Linux:

```
source venv/bin/activate
```

Once activated, you should see the virtual environment’s name in your terminal prompt.

Installing CrewAI

Next, we will install CrewAI and its tools using pip, the Python package manager:

```
pip install crewai
```

This command will download and install all the necessary packages for CrewAI. Depending on your CPU, this process might take a few minutes.

Verifying the Installation

To verify that CrewAI and its dependencies are installed correctly, you can list all installed packages:

```
pip list
```


You should see crewai in the list of installed packages.

Creating a New CrewAI Project

Now, let's create a new CrewAI project. We will use the CrewAI command-line interface (CLI) to do this:

```
crewai create buddy_ai
```

You will be prompted to enter a name for your project. In the video, the project is named buddy_ai, but you can choose any name you like.

Configuring the Project

After naming your project, you will be asked to select a provider and a model. In the video, OpenAI and the GPT-4 model are used:

1. Select OpenAI as the provider by entering 1.
2. Choose the GPT-4o model by entering 2.
3. Enter your OpenAI API access key. For now, you can enter a placeholder text, but remember to update it with a working API key when building the project.

Opening the Project

Finally, open the project in your preferred code editor. In the video, the command used is:

```
code .
```

This command opens the project in Visual Studio Code. You should see the project structure with folders like src, agents, config, and files like tasks.yaml and crew.py.

That's it! You have successfully set up your CrewAI project. In the next lessons, we will dive deeper into configuring and running our AI application.

Welcome back to our project! In this lesson, we will focus on setting up and configuring the initial agents and tasks for our application. We will be working with two main files: agents.yaml and tasks.yaml. These files are crucial for defining the roles, goals, and behaviors of our agents, as well as the tasks they will perform.

Updating agents.yaml

First, let's open the agents.yaml file. This file contains the definitions for our agents. Our goal is to update these agents or remove them and start from scratch. For this lesson, we will define two new agents: a PDF reader agent and a summarizer agent.

Defining the PDF Reader Agent

The PDF reader agent will be responsible for reading the content of a PDF file and extracting the text data. Here is how you can define it:

- **Role:** PDF Content Extractor
- **Goal:** Extract the entire text content from a fixed PDF document and provide it.
- **Backstory:** You are skilled at retrieving text from PDF documents. The PDF is located at [a specific path on your system].

Let's add the PDF reader agent to the agents.yaml file:

```
pdf_reader:
  role: >
    PDF Content Extractor
  goal: >
    Extract the entire text content from a fixed PDF document and provide it.
  backstory: >
    You are skilled at retrieving text from PDF documents.
    The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.
  llm: gpt-4o
  verbose: false
```

Defining the Summarizer Agent

The summarizer agent will take the extracted text from the PDF reader agent and summarize it into a concise and informative overview. Here is how you can define it:

- **Role:** AI Content Summarizer
- **Goal:** Summarize the given text into a concise and informative overview.
- **Backstory:** You specialize in taking long pieces of text and producing a clear, concise summary. Your summaries highlight the key points while maintaining factual accuracy.

Let's add the summarizer agent to the agents.yaml file:

```
summarizer:
  role: >
    AI Content Summarizer
  goal: >
    Summarize the given text into a concise and informative overview.
  backstory: >
    You specialize in taking long pieces of text and producing a clear, concise summa
```

```
ry.
    Your summaries highlight the key points while maintaining factual accuracy.
  llm: gpt-4o
  verbose: false
```

Updating tasks.yaml

Next, let's open the tasks.yaml file. This file contains the definitions for the tasks that our agents will perform. We will define two tasks: a reading task and a summarizing task.

Defining the Reading Task

The reading task will be assigned to the PDF reader agent. This task will involve extracting the entire text from the PDF document and returning the raw text content without modification. Here is how you can define it:

- **Description:** Extract the entire text from the PDF document at the specified location and return the raw text content without modification.
- **Expected Output:** The complete extracted text of the PDF document.
- **Agent:** pdf_reader

Let's add the reading task to the tasks.yaml file:

```
reading_task:
  description: >
    Extract the entire text from the PDF document at C:\Users\user\Downloads\Document
s\sample_document.pdf.
    Return the raw text content without modification.
  expected_output: >
    The complete extracted text of the PDF document.
  agent: pdf_reader
```

Defining the Summarizing Task

The summarizing task will be assigned to the summarizer agent. This task will involve taking the provided PDF text and summarizing it into a concise overview that highlights the main points. Here is how you can define it:

- **Description:** Take the provided PDF text and summarize it into a concise overview that highlights the main points.
- **Expected Output:** A concise summary (around 2-4 paragraphs) highlighting the key themes and facts from the document.
- **Agent:** summarizer
- **Context:** reading_task
- **Output File:** summary.md

Let's add the summarizing task to the tasks.yaml file:

```
summarizing_task:
  description: >
    Take the provided PDF text and summarize it into a concise overview that highligh
ts the main points.
```



```
  expected_output: >
    A concise summary (around 2-4 paragraphs) highlighting the key themes and facts f
rom the document.
  agent: summarizer
  context:
    - reading_task
  output_file: summary.md
```

Setting the Working API Access Key

Finally, we need to set the working API access key to use the GPT-4 model in our application. Open the .env file and add your OpenAI API key:

```
OPENAI_API_KEY=your_api_access_key_here
```

Replace your_api_access_key_here with your actual API key.

That’s it for this lesson! You have successfully defined the PDF reader and summarizer agents, as well as the reading and summarizing tasks. In the next lesson, we will continue building our application by adding more agents and tasks.

In this lesson, we will modify the main script and create instances of agents and tasks for our AI framework. This involves cleaning up the main script, defining agents and tasks, and ensuring that the framework can communicate with the created agents. Let’s go through the steps one by one.

Modifying the Main Script

First, we need to modify the main script to remove unnecessary comments and inputs. Here is how you can do it:

- Open the main script.
- Remove any comments that are no longer needed.
- In the run function, remove the inputs since the tasks are self-contained and agents know where the input is located.
- Add a comment stating that no inputs are needed because tasks are self-contained.
- In the train function and the test function, remove the line that takes inputs.
- Remove any tests that are present in the script.

Here is the modified main script:

```
def run():
    """
    Run the crew.
    """
    # No inputs needed since tasks are self-contained
    inputs = {}
    BuddyAi().crew().kickoff(inputs=inputs)

def train():
    """
    Train the crew for a given number of iterations.
    """
    inputs = {}
    try:
        BuddyAi().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2], i
nputs=inputs)
    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")
def test():
    """
    Test the crew execution and returns the results.
    """
    inputs = {}
    try:
        BuddyAi().crew().test(n_iterations=int(sys.argv[1]), openai_model_name=sys.ar
gv[2], inputs=inputs)
    except Exception as e:
        raise Exception(f"An error occurred while replaying the crew: {e}")
```

Creating Instances of Agents and Tasks

Next, we need to create instances of the agents and tasks so that the Crew AI framework can communicate with them. Follow these steps:

1. Open the crew.py script.
2. Remove any existing agent and task instances that were created during the initialization of the application.

Defining Agents

Let's define the agents first. We will create a PDF reader agent and a summarizer agent.

PDF Reader Agent

- Define the agent and return it.
- Specify the configuration from the agents.yaml file.
- Set the verbose property to True so that we can see how the agent is thinking and working.
- Use the PDFSearchTool to point to the fixed PDF document. We'll use a variable to store our PDF Path as well.

Here is the code for the PDF reader agent:

```
from crewai_tools import PDFSearchTool

PDF_PATH = 'C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf'

@agent
def pdf_reader(self) -> Agent:
    return Agent(
        config=self.agents_config['pdf_reader'],
        verbose=True,
        tools=[PDFSearchTool(pdf=PDF_PATH)]
    )
```

Summarizer Agent

- Define the agent and return it.
- Specify the configuration from the agents.yaml file.
- Set the verbose property to True.

Here is the code for the summarizer agent:

```
@agent
def summarizer(self) -> Agent:
    return Agent(
        config=self.agents_config['summarizer'],
        verbose=True
    )
```

Defining Tasks

Now, let's define the tasks. We will create a reading task and a summarizing task.

Reading Task

- Define the task and return it.
- Specify the configuration from the tasks.yaml file.
- Ensure that the task name matches the name in the tasks.yaml file.

Here is the code for the reading task:

```
@task
def reading_task(self) -> Task:
    return Task(
        config=self.tasks_config['reading_task'],
    )
```

Summarizing Task

- Define the task and return it.
- Specify the configuration from the tasks.yaml file.
- Ensure that the task name matches the name in the tasks.yaml file.
- Specify the output file for the summarization.

Here is the code for the summarizing task:

```
@task
def summarizing_task(self) -> Task:
    return Task(
        config=self.tasks_config['summarizing_task'],
        output_file='output/summary.md'
    )
```

Creating the Crew Instance

Finally, we need to create the crew instance that will contain the agents and tasks. Ensure that the process is sequential and that the verbose property is set to True so that we can see how the agents perform.

Here is the code for the crew instance:

```
@crew
def crew(self) -> Crew:
    return Crew(
        agents=self.agents,
        tasks=self.tasks,
        process=Process.sequential,
        verbose=True
    )
```

Installing Required Packages

If you encounter any syntactic errors or missing packages, make sure to install the required

packages. For example, if the crewai-tools package is not available, you can install it using the following command:

```
pip install crewai crewai-tools
```

By following these steps, you will have successfully modified the main script, created instances of agents and tasks, and ensured that the CrewAI framework can communicate with them. This sets the foundation for further development and integration of your AI application.

In this lesson, we will start building and testing our application using the **PDFReader** and **Summarizer** agents. These agents will help us extract text from a PDF document and summarize its content. Let's go through the steps to achieve this.

Prerequisites

- Ensure you have a PDF document available for testing. You can use a sample PDF provided in the project assets if you don't have one.
- Set up your development environment with the necessary tools and libraries.

Step 1: Prepare the PDF Document

First, place your PDF document in a specific directory. For this lesson, we will use the following path:

```
C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf
```

If you don't have a sample PDF, you can download one from the project assets. The sample document contains information about the CrewAI framework and its installation steps.

Step 2: Open the Terminal

Open your terminal and navigate to the project directory. Make sure you are in the **buddy_ai** folder, which contains the necessary configuration files and code for our application.

```
cd buddy_ai
```

Step 3: Run the Application

To build and run the application, use the following command:

```
crewai run
```

This command will create a virtual environment for the project and install all the required packages. This process might take a few minutes.

Step 4: Analyze the Output

Once the application starts running, the agents will begin their tasks:

1. **PDF Content Extractor** agent will look for the PDF document and return the raw text content without modification.
2. **AI Content Summarizer** agent will then summarize the extracted text.

The final output will be a summary of the PDF document's content.

Example Output

Here is an example of what the summarized output might look like:

```
The document provides a comprehensive guide on setting up and configuring AI projects
. It emphasizes the importance of organizing files logically. Overall, the guide serv
es as an essential resource for ensuring proper setup and efficient management.
```

Troubleshooting

- If the PDF document is not found, ensure the file path is correct and the document is accessible.
- If the agents fail to extract meaningful text, make sure the PDF contains searchable text and is not just images.

By following these steps, you should be able to successfully build and test your application using the **PDFReader** and **Summarizer** agents. In the next lessons, we will explore more advanced features and functionalities of our application.

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Welcome back! In this session, we will start working on another module of our project: the quiz generator. So far, we have implemented the PDFReader agent and the Summarizer agent. Now, we need to create an agent that generates quizzes based on the content and another agent that communicates with the user to test their knowledge using these quizzes.

Let's begin by defining new agents and their tasks.

Defining New Agents

Open your config folder and select the agents.yaml file. We will create two new agents: QuizMaker and StudyBuddy.

Step 1: Create the QuizMaker Agent

Copy and paste the following configuration for the QuizMaker agent:

The following code has been updated, and differs from the video:

```
quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm:
    provider: openai
    model: gpt-4o
  verbose: false
```

This agent will have the role of Quiz Creator and its goal will be to create a quiz based on the summarized text. The backstory describes the agent as an expert in formulating questions that test understanding.

Step 2: Create the StudyBuddy Agent

Next, copy and paste the following configuration for the StudyBuddy agent:

The following code has been updated, and differs from the video:

```
study_buddy:
  role: >
    Quiz Master
  goal: >
    Interactively test the user based on the quiz generated.
  backstory: >
    You are a helpful study buddy that asks questions to the user and checks their an
swer.
  llm:
    provider: openai
```



```
    model: gpt-4o
```

This agent will have the role of Quiz Master and its goal will be to interactively test the user based on the generated quiz. The backstory describes the agent as a helpful study buddy.

Defining New Tasks

Now that we have defined our new agents, we need to create tasks for them. Open the tasks.yaml file in the config folder.

Step 3: Create the QuizTask

Add the following configuration for the QuizTask:

```
quiz_task:
  description: >
    Using the summary of the PDF's content, create a quiz with multiple-
choice questions that test understanding of the main concepts.
    Provide say, 5 multiple-choice questions. Each question should have 4 options and
 indicate the correct answers.
  expected_output: >
    A quiz with 5 multiple-choice questions and their correct answers.
  agent: quiz_maker
  context:
    - summarizing_task
  output_file: quiz.md
```

This task will use the summary generated by the Summarizer agent to create a quiz with five multiple-choice questions. The expected output is a quiz with questions and their correct answers.

Step 4: Create the UserTestTask

Add the following configuration for the UserTestTask:

```
user_test_task:
  description: >
    Present the quiz questions to the user and ask them to provide their answers.
    After the user responds, check the correctness and provide a final score.
  expected_output: >
    A summary of user performance, e.g. "You answered X out of Y questions correctly.
"
  agent: study_buddy
  context:
    - quiz_task
  human_input: true
```

This task will present the quiz questions to the user and ask for their answers. The agent will then check the correctness of the answers and provide a final score. The human_input: true property indicates that this task will interact with the user.

Saving the Configuration

After adding the new agents and tasks, save the agents.yaml and tasks.yaml files.

Summary

In this lesson, we defined two new agents: QuizMaker and StudyBuddy. We also created tasks for these agents to generate a quiz based on summarized content and to interactively test the user's knowledge. These agents and tasks work sequentially, waiting for each other to complete their respective tasks.

In the next lesson, we will implement the logic for these tasks and agents to bring our quiz generator module to life.

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will continue building our application by adding two new agents: QuizMaker and StudyBuddy. We will also create tasks for quiz generation and user testing. Let's dive into the steps required to achieve this.

Adding New Agents

To add the new agents, follow these steps:

1. Open the crew.py script.
2. Locate the agent configuration section.
3. Duplicate an existing agent configuration template and modify it for the new agents.

Here is how you can configure the QuizMaker agent:

The following code has been updated, and differs from the video:

```
@agent
def quiz_maker(self) -> Agent:
    return Agent(
        config=self.agents_config['quiz_maker'],
        verbose=False
    )
```

And here is the configuration for the StudyBuddy agent:

```
@agent
def study_buddy(self) -> Agent:
    return Agent(
        config=self.agents_config['study_buddy'],
        verbose=False,
    )
```

Note that the strict_parser parameter is no longer needed in more recent versions of CrewAI and therefore can be safely removed from the code.

Creating New Tasks

Next, we need to create tasks for quiz generation and user testing. We'll start with the quiz task, which can be added by copying and pasting any task from the list and reconfiguring it for the quiz_task we made last lesson:

```
@task
def quiz_task(self) -> Task:
  return Task(
    config=self.tasks_config['quiz_task'],
  )
```


Implementing the Reading Logic

In order to create the user task, we first need to update our reading task. As before in this task, we're going to extract the content from a PDF file. However, we want to add some more logic by doing the follow:

1. **Verify the file path and access**: We check if the PDF file exists and can be accessed at the given path. If not, we return an error message. This helps us prevent program crashes.
2. **Attempt multiple queries to extract content**: We try different queries to extract the content from the PDF. We use a list of queries and loop through each one. For each query, we use the Search a PDF's content tool to get the result. If we get a result that's not the fallback snippet, we store it as the extracted content and break out of the loop.
3. **Confirm content extraction**: If we didn't extract any content, we return an error message indicating that we couldn't extract meaningful text from the PDF.

The code uses the os module to check if the file exists and is accessible. It also uses the agent object to interact with the Search a PDF's content tool.

```
@task
def reading_task(self) -> Task:
    """
    This task will:
    - verify file existence and access
    - attempt multiple queries to extract full content
    - Return extracted content or clear out all error messages
    """
    def reading_logic(agent, inputs):
        # 0. verify the file path and access
        if not os.path.isfile(PDF_PATH):
            return "ERROR: PDF file not found or not accessible at the given path."

        # 1. Attempt to retrieve content from the PDF using different queries.
        queries = ["", "all content", "document", "*"]
        extracted_content = None
        fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files
."

        for q in queries:
            agent.think(f"Trying query '{q}' to get PDF content. ")
            result = agent.use_tool("Search a PDF's content", {"query": q})
            if result and fallback_snippet not in result:
                # trying to get something different from the fallback description
                extracted_content = result.strip()
                break

        # 2. confirm that content was extracted
        if not extracted_content:
            return (
                "ERROR: unable to extract meaningful text from the PDF."
                "Make sure that PDF has searchable text or try another document."
            )

        return extracted_content
```



```
    return Task(
        config=self.tasks_config['reading_task'],
        logic=reading_logic
    )
```

Summary

In this lesson, we added two new agents, QuizMaker and StudyBuddy, and created a task for quiz generation. We also implemented the reading logic to verify file existence, access the file, and extract content using queries. This sets the foundation for further enhancing our application’s capabilities.

In this lesson, we will continue building our PDF processing tool by adding two new tasks: summarizing the extracted PDF text and creating a user test task to handle user interactions for a quiz. Let's break down the steps involved in these processes.

Summarizing Task

While we already have a summarizing task, like last lesson, we need to make some updates so this works with our quiz module. The new summarizing task will take the extracted PDF text and generate a concise summary as before, but will also do some error checking to make sure we actually got text that can be summarized.

Let's start by defining the summarizing logic. We need to handle any errors that might occur during the reading task. If the reading task returns an error, we should gracefully handle it and provide a clear message.

```
@task
def summarizing_task(self) -> Task:
  """
    Summarize the extracted PDF text.
      If the reading_task returned an ERROR,
        handle that gracefully here!
  """
  def summarizing_logic(agent, inputs, context):
    pdf_text = context.get('reading_task', '')
    if pdf_text.startswith("ERROR:"):
      return f"Cannot summarize because: {pdf_text}"

    return agent.llm(
      f"Summarize the following text into a concise overview:\n\n{pdf_text}"
    ).strip
  return Task(
    config=self.tasks_config['summarizing_task'],
    logic=summarizing_logic,
    output_file='output/summary.md'
  )
```

In the code above:

- pdf_text retrieves the text from the context of the reading task.
- If the text starts with "ERROR:", it returns a message indicating that summarization cannot be performed.
- Otherwise, it uses the language model to generate a summary of the text.

User Test Task

The user test task will present quiz questions to the user, collect their answers, and provide a score based on their performance. Here's how we can achieve this:

- Create a test logic that retrieves the quiz text from the context.
- Prompt the user to provide their answers in a specific format.
- Parse the user's answers and compare them with the correct answers to calculate the score.

Let's start by defining the test logic. We need to retrieve the quiz text from the context of the quiz

task and prompt the user to provide their answers.

```
@task
def test_logic(agent, inputs, context):
    quiz_text = context.get('quiz_task', '')

    # The agent will prompt the user:
    agent.think("I will now ask the user to answer the quiz.")
    agent.say("Here are your quiz questions:n")
    agent.say(quiz_text)
    agent.say("Please provide your answers in the format: A,B,C,... for each question
 in order.")

    user_answers = inputs.get('human_input')
    user_answers_list = [ans.strip().upper() for ans in user_answers.split(",")]
```

In the code above:

- quiz_text retrieves the quiz text from the context of the quiz task.
- The agent prompts the user to provide their answers in a specific format.
- user_answers retrieves the human input, and user_answers_list parses the user's answers.

Next, we need to extract the correct answers from the quiz text and compare them with the user's answers to calculate the score.

```
import re
correct_answers = re.findall(r'Correct Answer:s*([A-D])', quiz_text, re.IGNORECASE)

score = 0
for user_ans, correct_ans in zip(user_answers_list, correct_answers):
    if user_ans == correct_ans.upper():
        score += 1

total = len(correct_answers)
result = f"You answered {score} out of {total} correctly!"

agent.say(result)
return result
```

In the code above:

- We use a regular expression to find all correct answers in the quiz text.
- We initialize the score to 0 and iterate through the user's answers and correct answers.
- If the user's answer matches the correct answer, we increment the score.
- Finally, we calculate the total number of correct answers and provide the result to the user.

Updating the Crew Configuration

Now that we have defined the test logic, we need to update our crew configuration to return the task

outside the function we just created.

```
@task
...
    return Task(
        config=self.tasks_config['user_test_task'],
        logic=test_logic
    )
```

With these updates, our PDF processing tool can now summarize the extracted PDF text and handle user interactions for a quiz. In the next lesson, we will test our quiz and make sure it is functioning as intended!

Welcome back! In this session, we will build and test our application to utilize the newly created Quiz Generator feature. By the end of this lesson, you will have run the application and interacted with the quiz generator through the terminal.

Navigating to the Project Directory

First, ensure you are in the correct directory. Open your terminal and navigate to the buddy_ai folder using the following command:

```
cd buddy_ai
```

Running the Application

Once you are in the correct directory, you can run the application with the following command:

```
crewai run
```

This command will start the application. If there are any configuration errors, they will be displayed in the terminal.

Handling Errors

If you encounter an error, it might be due to a misconfiguration in the tasks.yaml file. Let’s examine a common issue and how to fix it.

Checking the Configuration File

Open the tasks.yaml file located in the config folder. Ensure that the tasks are correctly defined. Here is an example of what the tasks.yaml file should look like:

```
reading_task:
  description: >
    Extract the entire text from the PDF document at C:\Users\user\Downloads\Document
s\sample_document.pdf.
    Return the raw text content without modification.
  expected_output: >
    The complete extracted text of the PDF document.
  agent: pdf_reader

summarizing_task:
  description: >
    Take the provided PDF text and summarize it into a concise overview that highligh
ts the main points.
  expected_output: >
    A concise summary (around 2-4 paragraphs) highlighting the key themes and facts f
rom the document.
  agent: summarizer
  context:
    - reading_task
  output_file: summary.md
```



```
quiz_task:
  description: >
    Using the summary of the PDF's content, create a quiz with multiple-
choice questions that test understanding of the main concepts.
    Provide say, 5 multiple-choice questions. Each question should have 4 options and
 indicate the correct answers.
  expected_output: >
    A quiz with 5 multiple-choice questions and their correct answers.
  agent: quiz_maker
  context:
    - summarizing_task
  output_file: quiz.md

user_test_task:
  description: >
    Present the quiz questions to the user and ask them to provide their answers.
    After the user responds, check the correctness and provide a final score.
  expected_output: >
    A summary of user performance, e.g. "You answered X out of Y questions correctly.
"
  agent: study_buddy
  context:
    - quiz_task
  human_input: true
```

If you encounter a syntax error, carefully check the YAML file for any missing dashes or incorrect indentation.

Running the Application Again

After fixing any errors, clear the terminal and run the application again:

```
crewai run
```

If everything is configured correctly, you should see the output from the agents and tasks.

Interacting with the Quiz Generator

Once the application is running, you will be prompted to provide feedback on the final result. You can respond with “looks good” or a similar phrase when you are satisfied with the output.

The quiz generator will then present you with multiple-choice questions. Provide your answers in the format A, B, C, ... for each question in order.

Reviewing Your Answers

After submitting your answers, the application will compare them to the correct answers and provide your score. For example:

```
You answered 3 out of 5 questions correctly.
```


You can review the correct answers and try again if needed.

Exiting the Application

To exit the application, you can type “looks good” or “exit” in the terminal.

Summary

In this lesson, you learned how to run the application, handle configuration errors, and interact with the quiz generator through the terminal. By following these steps, you should be able to test the Quiz Generator feature and review your understanding of the material.

In the next lesson, we will explore more advanced features and configurations of the application.

Welcome back to our StudyBuddy AI project! In this session, we will focus on creating the flashcard mechanism. This involves expanding our current user feedback system to handle new commands like show and flip for flashcards, and win to parse and process data accordingly. We will create five flashcards, each with a front (question or term) and a back (explanation). Let's dive into the steps to achieve this.

Creating the FlashcardMaker Agent

To create flashcards, we need a new agent called FlashcardMaker. This agent will use summarized text and turn it into question-answer pairs, representing flashcards. Let's start by configuring this agent in the agents.yaml file.

Configuring the FlashcardMaker Agent

Open the agents.yaml file and scroll down to add the new agent configuration. Here is how you can set it up:

```
flashcard_maker:
  role: >
    Flashcard Generator
  goal: >
    Create flashcards from the summarized text to help the user study.
  backstory: >
    You turn summarized text into helpful question-
answer pairs, each representing a flashcard.
  llm: gpt-4o
  verbose: false
  strict_parser: false
  function_calling_llm: null
```

Explanation of the configuration:

- role: Defines the role of the agent as a Flashcard Generator.
- goal: Specifies the goal of the agent, which is to create flashcards from summarized text.
- backstory: Provides context for the agent's function.
- llm: Sets the language model to gpt-4o.
- verbose: Set to false to avoid detailed output.
- strict_parser: Set to false for flexibility in parsing.
- function_calling_llm: Set to null as we do not need a specialized secondary model.

Applying Configuration to All Agents

We need to apply the function_calling_llm: null configuration to all agents. This is because our agents use GPT models that natively support function calling, and we do not need a specialized secondary model. Here is how you can update the other agents:

```
pdf_reader:
  ...
  function_calling_llm: null

summarizer:
  ...
  function_calling_llm: null
```



```
quiz_maker:
  ...
  function_calling_llm: null

study_buddy:
  ...
  function_calling_llm: null
```

Defining Tasks for the Agents

Next, we need to define two tasks: one for the FlashcardMaker agent and another for the StudyBuddy agent. These tasks will be executed sequentially.

Task 1: Flashcard Task

This task will create flashcards using the summarized text. Add the following configuration to the tasks.yaml file:

```
flashcard_task:
  description: >
    Using the summary of the PDF's content, create 5 flashcards. Each card has a "fro
nt"
    (question or term) and a "back" (explanation).
  expected_output: >
    A structured list or text specifying each flashcard's front and back.
  agent: flashcard_maker
  context:
    - summarizing_task
  output_file: flashcards.md
```

Task 2: Study Flashcards Task

This task will interact with the user to go through the flashcards. Add the following configuration to the tasks.yaml file:

```
study_flashcards_task:
  description: >
    Go through the flashcards one by one with the user. The user can type "show" to d
isplay a card's front,
    "flip" to reveal its back, or "quit" to exit.
  expected_output: >
    Interactive session with the user. No final text is required, just user engagemen
t.
  agent: study_buddy
  context:
    - flashcard_task
  human_input: true
```

Summary

In this lesson, we configured the FlashcardMaker agent and defined two tasks to create and study flashcards. We also ensured that all agents have the function_calling_llm set to null for simplicity and cost efficiency. This setup will allow our StudyBuddy AI to create and interact with flashcards effectively.

In the next lesson, we will dive deeper into implementing the interaction logic for the flashcards. Stay tuned!

In this lesson, we will focus on creating agent instances and task instances in our Python code. We will be working within the crew.py file to define these agents and tasks. By the end of this lesson, you will have a clear understanding of how to set up agents and tasks, and how to implement the main logic for a specific task.

Defining Agents

Let's start by defining our agent. The FlashcardMaker agent is responsible for creating flashcards. Here is how you can define it:

```
  @agent
  def flashcard_maker(self) -> Agent:
    return Agent(
      config=self.agents_config['flashcard_maker'],
      verbose=False,
      strict_parser=False,
      function_calling_llm=None
    )
```

Defining Tasks

Next, we will define the tasks that these agents will perform. We will define two tasks: flashcard_task and study_flashcards_task.

Flashcard Task

The flashcard_task is responsible for generating flashcards. Here is how you can define it as most aspects are covered in our prompt:

```
def flashcard_task(self) -> Task:
    return Task(
        config=self.tasks_config['flashcard_task']
    )
```

Study Flashcards Task

The study_flashcards_task is responsible for interacting with the user during the study session. This task will have a specific logic for studying flashcards. Here is how you can define it to start so we can get our basic configurations in:

```
def study_flashcards_task(self) -> Task:
    def study_logic(agent, inputs, context):


    return Task(
        config=self.tasks_config['study_flashcards_task'],
        logic=study_logic,
        human_input=True
    )
```

Implementing the Main Logic

Now that we have defined our agents and tasks, we can implement the main logic for the study_flashcards_task. This logic will handle the interaction with the user during the study session.

The main logic for the study_flashcards_task involves the following steps:

1. Parse the flashcards from the output of the flashcard_task.
2. Interact with the user to go through each flashcard.
3. Handle user inputs such as ‘show’, ‘flip’, and ‘quit’ to navigate through the flashcards.

By following these steps, you will be able to create a interactive study session using flashcards. This lesson has covered the basics of defining agents and tasks. In the next lesson, we will implement the main logic described above so our study flashcards task works.

In this lesson, we will focus on parsing flashcards from the output of a flashcard task and implementing a simple user interface to navigate through them. This involves letting the user type commands like ‘show’, ‘flip’, or ‘quit’ to interact with the flashcards. Let’s break down the steps involved in achieving this.

Parsing Flashcards

The first step is to parse the flashcards from the output of the flashcard task. We need to retrieve the data and handle any potential errors.

```
flashcards_text = context.get('flashcards_task', '')
if not flashcards_text.strip():
    return "No flashcards found. Possibly an error."
```

Agent Interaction

Next, we use the agent to think and parse the flashcards from the text.

```
agent.think("Parsing flashcards from text...")
```

Storing Flashcards

We assume the agent returns the flashcards in a specific format. For simplicity, let’s store them line by line. We will use a one-liner algorithm to split the text into lines and strip any whitespace.

```
lines = [l.strip() for l in flashcards_text.split('\n') if l.strip()]
```

Building the Flashcard Structure

We need to build a list of dictionaries, where each dictionary represents a flashcard with ‘front’ and ‘back’ keys. We will iterate through the lines and use simple matching to extract the front and back of each flashcard.

```
flashcards = []
current_fc = {}

for line in lines:
    if line.lower().startswith("front:"):
        current_fc = {"front": line[6:].strip(), "back": ""}
    elif line.lower().startswith("back:"):
        current_fc["back"] = line[5:].strip()
        flashcards.append(current_fc)
        current_fc = {}

if not flashcards:
    return "Couldn't parse any flashcards from the text."
```

User Interaction

Now that we have the flashcards parsed and stored, we can implement the user interaction. We will use an index variable to keep track of the current flashcard and a flag to indicate whether the front of the flashcard is showing.

```
agent.say(f"I have {len(flashcards)} flashcards loaded. We'll go through them now.n")
agent.say("Type 'show' to see the next flashcard, 'flip' to see its back, or 'quit' t
o end.n")

index = 0
showing_front = False
user_input = inputs.get('human_input', '').lower()
```

Handling User Commands

We will handle the user commands in a simple state machine. The user can type ‘show’, ‘flip’, or ‘quit’ to navigate through the flashcards.

- **Quit Command:** If the user types ‘quit’, we exit the flashcard study session.
- **Show Command:** If the user types ‘show’, we display the front of the current flashcard.
- **Flip Command:** If the user types ‘flip’, we display the back of the current flashcard.

```
if user_input == "quit":
    return "Exiting flashcard study session."

if index >= len(flashcards):
    return "No more flashcards left."

if user_input == "show":
    agent.say(f"Flashcard {index + 1} front: {flashcards[index]['front']}")
    showing_front = True
    return "Type 'flip' next time to see the back, or 'quit' to stop."
elif user_input == "flip" and not showing_front:
    return "You need to 'show' the card first. Then you can 'flip' it."
elif user_input == "flip" and showing_front:
    agent.say(f"Flashcard {index+1} back: {flashcards[index]['back']}")
    return f"Now you can type 'show' to move to the next card or 'quit' to stop."
else:
    return "Invalid command. Please type 'show' or 'flip' or 'quit'"
```

Summary

In this lesson, we learned how to parse flashcards from the output of a flashcard task and implement a simple user interface to navigate through them. We used an agent to think and parse the flashcards, stored them in a list of dictionaries, and handled user commands to display the front and back of each flashcard. This provides a basic framework for interacting with flashcards in a study session.

Welcome to our final lesson on our flashcard module. With everything set up, we can now try running our module.

Getting Started

To begin, ensure you have your terminal open and navigate to the budddy_ai folder. This folder will contain the necessary files and scripts for your project.

Running the Project

1. In your terminal, type the following command to start the project:

   ```
   crewai run
   ```

2. This command will initiate the PDF content extraction process. You should see the content being extracted and summarized.

Generating Quiz Questions

Once the content is summarized, the QuizBot Creator agent will generate multiple-choice questions. Here’s how you can interact with this feature:

- The agent will create 5 multiple-choice questions along with their answers.
- You can test the quiz by providing random answers to see how the system responds.

Interacting with the State Machine

The agent will then create a state machine for the user, allowing you to input answers to the questions. Here’s what you can expect:

- You can input random answers to test the system.
- The system will provide feedback based on your answers, encouraging you to keep practicing to improve your score.

Generating Flashcards

After providing feedback, the system will generate flashcards. Here’s how you can interact with the flashcards:

- Type show to display the front of the flashcard.
- Type flip to reveal the back of the flashcard.
- Continue using show and flip commands to navigate through the flashcards.

Key Customizable Elements

The key customizable elements in a CrewAI project include:

- Agents and tasks configuration
- Crew logic
- Environment variables

Exiting the Application

To exit the flashcard mechanism process, type quit. You will receive a thank you message for engaging with the flashcard session. To exit the entire application, type looks good.

Conclusion

Congratulations! You have successfully set up and interacted with a CrewAI project. This guide has covered the basics of running the project, generating quiz questions, interacting with the state machine, generating flashcards, and exiting the application. As you become more comfortable with these steps, you can explore further customization and advanced features offered by CrewAI.

Happy learning, and welcome to the world of CrewAI at Zenva!

Welcome back to our learning journey at Zenva! In this session, we have an exciting challenge tailored just for you. This task will help you reinforce your understanding of configuring applications and managing output data. Let’s dive right in!

Challenge Overview

In our previous session, while running the agents, you might have noticed that our terminal displayed unnecessary messages. For example, when creating quizzes, the answers were also shown alongside the questions. This clutter makes it difficult to focus on the essential information.

Objective

Your main task is to configure the application so that only the required information is displayed in the terminal. Currently, the agents are printing all the generated data, which includes both necessary and unnecessary information.

Steps to Complete the Challenge

1. **Identify the Problem:** Recognize the unnecessary messages that are being displayed in the terminal.
2. **Review Configurations:** Look back at the configurations we did in the previous sessions. Pay close attention to the details, as they hold the key to solving this challenge.
3. **Modify the Configuration:** Adjust the settings to ensure that only the required information is printed to the terminal.
4. **Test the Changes:** Run the application again to verify that the output is now clean and free of unnecessary data.

Tips for Success

- **Focus on Details:** Small adjustments in the configuration can make a big difference in the output.
- **Use Comments:** Adding comments to your code can help you understand the purpose of each configuration setting.
- **Seek Help:** If you encounter any difficulties, don’t hesitate to reach out to your peers or instructors for guidance.

By completing this challenge, you will gain a deeper understanding of how to manage and configure application outputs effectively. This skill is crucial for creating clean and efficient applications. Good luck, and remember, every challenge is an opportunity to learn and grow!

In this lesson, we will complete the challenge by making necessary adjustments to our agents and ensuring that the quiz answers are not displayed to the user. This will involve modifying the verbosity settings of our agents and updating the goal of the quiz maker agent. Let's go through the steps to achieve this.

Adjusting Agent Verbosity

The first step is to adjust the verbosity of our agents. Verbosity controls the amount of detail the agents provide in their outputs. By setting verbosity to false, we can ensure that only the necessary information is displayed to the user.

Let's start by modifying the agents.yaml file. This file contains the configuration for our agents.

Open the agents.yaml file and locate the following agents:

- summarizer
- quiz_maker
- study_buddy

Change the verbose property from true to false for each of these agents. Here is the updated configuration:

```
summarizer:
  role: >
    AI Content Summarizer
  goal: >
    Summarize the given text into a concise and informative overview.
  backstory: >
    You specialize in taking long pieces of text and producing a clear, concise summa
ry.
    Your summaries highlight the key points while maintaining factual accuracy.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null

quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null

study_buddy:
  role: >
    Quiz Master
  goal: >
    Interactively test the user based on the quiz generated.
  backstory: >
    You are a helpful study buddy that asks questions to the user and checks their an
swer.
  llm: gpt-4o
```



```
  verbose: false
  function_calling_llm: null
```

Next, open the crew.py file and ensure that the verbosity settings are correctly applied to the agents. The relevant part of the code should look like this:

```
@agent
def summarizer(self) -> Agent:
    return Agent(
        config=self.agents_config['summarizer'],
        verbose=False
    )

@agent
def quiz_maker(self) -> Agent:
    return Agent(
        config=self.agents_config['quiz_maker'],
        verbose=False,
        strict_parser=False
    )

@agent
def study_buddy(self) -> Agent:
    return Agent(
        config=self.agents_config['study_buddy'],
        verbose=False,
    )
```

Updating the Quiz Maker Goal

The next step is to update the goal of the quiz maker agent to ensure that the answers to the quiz questions are not shown to the user. Open the agents.yaml file and locate the quiz_maker agent. Update the goal property as follows:

```
quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
    Do not show the answers of the questions to the user.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null
```

Running the Application

After making these changes, save all the files and open your terminal. Navigate to the directory containing your project and run the following command:


```
crewai run
```

This command will start the application and apply the changes we made. You should see the application extracting data from the PDF file and generating a quiz. Notice that the answers to the quiz questions are not displayed, as intended.

Once the quiz is generated, the Quizmaster will prompt you to provide answers to the questions. You can input your answers in the format A, B, C, ... for each question in order. After submitting your answers, the Quizmaster will check them and provide a score.

If you want to start a flashcard session, you can type looks good. The application will then guide you through the flashcards one by one. You can type show to see the front of a flashcard, flip to reveal its back, or quit to exit the session.

To close the application, type quit followed by looks good.

Conclusion

In this lesson, we adjusted the verbosity settings of our agents and updated the goal of the quiz maker agent to ensure that the answers to the quiz questions are not shown to the user. We also ran the application to see the changes in action.

Welcome back to our course on integrating local large language models into our StudyBuddy AI application. In this lesson, we will focus on using Ollama to download and run large language models locally on our development machines. Ollama provides an easy-to-use API for managing and running these models, which we will utilize to enhance our application.

Getting Started with Ollama

To begin, you need to download the Ollama installer. Follow these steps:

1. Open your web browser and navigate to ollama.com.
2. On the homepage, click the download button to get the installer.
3. For more information, you can visit the Ollama GitHub page, which provides details on the available models and how to use them in your projects.

Downloading and Running a Model

Once you have the installer, you can download and run a model. For this demonstration, we will use the Llama 3.2 model. Here's how you can do it:

1. Open your terminal and navigate to the directory where you want to run Ollama.
2. To see the available commands, type:

```
ollama --help
```

3. To list the available models, type:

```
ollama list
```

4. To start the Ollama service, type:

```
ollama serve
```

Updating the Agents Configuration

Next, we need to update our agents.yaml file to use the Llama 3.2 model. Open the agents.yaml file and update the llm field for each agent to llama-3.2.

```
pdf_reader:
  role: >
    PDF Content Extractor
  goal: >
    Extract the entire text content from a fixed PDF document and provide it.
    Use Llama 3.2 model.
  backstory: >
    You are skilled at retrieving text from PDF documents.
    The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

summarizer:
  role: >
    AI Content Summarizer
  goal: >
```

```
    Summarize the given text into a concise and informative overview.
  backstory: >
    You specialize in taking long pieces of text and producing a clear, concise summa
ry.
    Your summaries highlight the key points while maintaining factual accuracy.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
    Do not show the answers of the questions to the user.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

study_buddy:
  role: >
    Quiz Master
  goal: >
    Interactively test the user based on the quiz generated.
  backstory: >
    You are a helpful study buddy that asks questions to the user and checks their an
swer.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

flashcard_maker:
  role: >
    Flashcard Generator
  goal: >
    Create flashcards from the summarized text to help the user study.
  backstory: >
    You turn summarized text into helpful question-
answer pairs, each representing a flashcard.
  llm: llama-3.2
  verbose: false
  strict_parser: false
  function_calling_llm: null
```

Updating the Crew Script

Now, let’s update the crew.py script to integrate the local large language model. Open the crew.py file and make the following changes:

1. Import the necessary class from the crew.ai package:

```
from crewai import LLM
```

2. Specify the local address, model, and base URL for each agent. For example, for the pdf_reader agent:

```
def pdf_reader(self) -> Agent:
    return Agent(
        config=self.agents_config['pdf_reader'],
        llm=LLM(
            model="ollama/llama3.2",
            base_url="http://localhost:11434"
        ),
        verbose=False,
        tools=[PDFSearchTool(pdf=PDF_PATH)]
    )
```

3. Repeat the above step for all other agents (summarizer, quiz_maker, study_buddy, and flashcard_maker).

```
@agent
  def summarizer(self) -> Agent:
    return Agent(
      config=self.agents_config['summarizer'],
      llm=LLM(
        model="ollama/llama3.2",
        base_url="http://localhost:11434"
      ),
      verbose=False
    )

  @agent
  def quiz_maker(self) -> Agent:
    return Agent(
      config=self.agents_config['quiz_maker'],
      llm=LLM(
        model="ollama/llama3.2",
        base_url="http://localhost:11434"
      ),
      verbose=False,
      strict_parser=False
    )

  @agent
  def study_buddy(self) -> Agent:
    return Agent(
      config=self.agents_config['study_buddy'],
      llm=LLM(
        model="ollama/llama3.2",
        base_url="http://localhost:11434"
      ),
      verbose=False,
    )

  @agent
  def flashcard_maker(self) -> Agent:
```

```
    return Agent(
      config=self.agents_config['flashcard_maker'],
      llm=LLM(
        model="ollama/llama3.2",
        base_url="http://localhost:11434"
      ),
      verbose=False,
      strict_parser=False,
      function_calling_llm=None
    )
```

Modifying the Reading Logic

We also need to modify the reading logic in the crew.py script. Since we know the PDF file contains text data, we can simplify the query list and remove the loop. Update the reading_task as follows:

```
@task
def reading_task(self) -> Task:
    """
        This task will:
        - verify file existence and access
        - attempt multiple queries to extract full content
        - Return extracted content or clear out all error messages
    """
    def reading_logic(agent, inputs):
        # 0. verify the file path and access
        if not os.path.isfile(PDF_PATH):
            return "ERROR: PDF file not found or not accessible at the given path."

        # 1. Attempt to retrieve content from the PDF using different queries.
        # queries = ["", "all content", "document", "*"]
        extracted_content = None
        fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files
."

        # for q in queries:
        agent.think(f"Trying query (all content) to get PDF content. ")
        result = agent.use_tool("Search a PDF's content", {"query": "all content"})
        if result and fallback_snippet not in result:
            # trying to get something different from the fallback description
            extracted_content = result.strip()

        # 2. confirm that content was extracted
        if not extracted_content:
            return (
                "ERROR: unable to extract meaningful text from the PDF."
                "Make sure that PDF has searchable text or try another document."
            )

        return extracted_content

    return Task(
        config=self.tasks_config['reading_task'],
```

```
        logic=reading_logic
    )
```

Running the Application

Before building the Crew AI application, run the following command to test if the Llama 3.2 model is running:

```
ollama run llama-3.2
```

If the model is running correctly, you should see a response. Now, you can build your project by running:

```
crewai run
```

Conclusion

In this lesson, we integrated a local large language model into our StudyBuddy AI application using Ollama. We updated the agents configuration, modified the crew script, and adjusted the reading logic to work with the local model. By following these steps, you can enhance your application with powerful language models running locally on your development machine.

Remember, for more complex tasks, you might need a more powerful model or a compatible graphics card. Explore different models and fine-tune them to suit your specific needs.

That’s it for this session. Happy coding!

In this project, we integrated multiple specialized agents to manage tasks within Crew.ai. This approach allowed us to leverage different agents for specific tasks, such as reading PDFs, summarizing content, generating quizzes, and handling user interactions. Crew.ai orchestrated each step of the process, from text extraction to quiz creation, ensuring that the workflow was organized and easily modifiable. By incorporating interactive learning concepts, we transformed static PDF text into dynamic quizzes and flashcards that provide immediate user feedback. This modular system ensures that each agent focuses on its designated task, making the system both modular and straightforward to upgrade.

Transforming PDFs into Interactive Learning Experiences

With the completion of your StudyBuddy application, you have successfully turned PDFs into interactive learning experiences. However, the journey doesn’t end here. There are numerous avenues to explore and enhance your application further.

Potential Enhancements

- **Web Interface Design:** Create a sleek and user-friendly web interface to make the learning experience more engaging.
- **Spaced Repetition:** Implement spaced repetition techniques to improve long-term retention and attention.
- **Hierarchical Task Processing:** Switch to hierarchical task processing for more complex and efficient interactions.
- **Advanced Memory and Caching:** Explore advanced memory and caching techniques to optimize performance.
- **Embeddings:** Incorporate embeddings to make your AI even smarter and more capable of understanding context.

Continuing Your Learning Journey

The possibilities for enhancing your StudyBuddy application are endless. Stay curious and keep learning to build unique and fun projects. If you wish to continue exploring project-based courses, consider enrolling in Zenva Academy.

About Zenva Academy

Zenva Academy is an online learning platform with over a million students. It offers a wide range of courses suitable for beginners and those looking to learn something new. The courses are versatile, allowing you to learn in a way that suits your preferences.

1. **Diverse Course Offerings:** Zenva Academy features a variety of courses covering different topics and skill levels.
2. **Flexible Learning:** The courses are designed to be flexible, allowing you to learn at your own pace and in your preferred style.
3. **Community Support:** Join a community of learners and gain support from instructors and fellow students.

Once again, thank you for joining us on this learning journey. We hope to see you in future projects and courses on Zenva Academy.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Complete Project with OpenAI

agents.yaml

Found in folder: **/src/buddy_ai/config**

This code defines five roles for a language model: PDF Content Extractor, AI Content Summarizer, Quiz Creator, Quiz Master, and Flashcard Generator. Each role has a specific goal, backstory, and configuration settings, such as the language model to use and verbosity level. These roles are used to automate tasks in a study aid tool.

```
pdf_reader:
  role: >
    PDF Content Extractor
  goal: >
    Extract the entire text content from a fixed PDF document and provide it.
  backstory: >
    You are skilled at retrieving text from PDF documents.
    The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null

summarizer:
  role: >
    AI Content Summarizer
  goal: >
    Summarize the given text into a concise and informative overview.
  backstory: >
    You specialize in taking long pieces of text and producing a clear, concise summa
ry.
    Your summaries highlight the key points while maintaining factual accuracy.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null

quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
    Do not show the answers of the questions to the user.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null
```



```
study_buddy:
  role: >
    Quiz Master
  goal: >
    Interactively test the user based on the quiz generated.
  backstory: >
    You are a helpful study buddy that asks questions to the user and checks their an
swer.
  llm: gpt-4o
  verbose: false
  function_calling_llm: null

flashcard_maker:
  role: >
    Flashcard Generator
  goal: >
    Create flashcards from the summarized text to help the user study.
  backstory: >
    You turn summarized text into helpful question-
answer pairs, each representing a flashcard.
  llm: gpt-4o
  verbose: false
  strict_parser: false
  function_calling_llm: null
```

tasks.yaml

Found in folder: **/src/buddy_ai/config**

This code defines a series of tasks to process a PDF document, including extracting text, summarizing content, generating quizzes and flashcards, and interacting with the user to test their understanding. Each task specifies an agent responsible for executing the task, the expected output, and any required context or human input. The tasks are defined in a YAML file, which provides a structured format for describing the workflow.

```
reading_task:
  description: >
    Extract the entire text from the PDF document at C:\Users\user\Downloads\Document
s\sample_document.pdf.
    Return the raw text content without modification.
  expected_output: >
    The complete extracted text of the PDF document.
  agent: pdf_reader

summarizing_task:
  description: >
    Take the provided PDF text and summarize it into a concise overview that highligh
ts the main points.
  expected_output: >
    A concise summary (around 2-4 paragraphs) highlighting the key themes and facts f
rom the document.
  agent: summarizer
  context:
```

```
    - reading_task
  output_file: summary.md

quiz_task:
  description: >
    Using the summary of the PDF's content, create a quiz with multiple-
choice questions that test understanding of the main concepts.
    Provide say, 5 multiple-choice questions. Each question should have 4 options and
 indicate the correct answers.
  expected_output: >
    A quiz with 5 multiple-choice questions and their correct answers.
  agent: quiz_maker
  context:
    - summarizing_task
  output_file: quiz.md

user_test_task:
  description: >
    Present the quiz questions to the user and ask them to provide their answers.
    After the user responds, check the correctness and provide a final score.
  expected_output: >
    A summary of user performance, e.g. "You answered X out of Y questions correctly.
"
  agent: study_buddy
  context:
    - quiz_task
  human_input: true

flashcard_task:
  description: >
    Using the summary of the PDF's content, create 5 flashcards. Each card has a "fro
nt"
    (question or term) and a "back" (explanation).
  expected_output: >
    A structured list or text specifying each flashcard's front and back.
  agent: flashcard_maker
  context:
    - summarizing_task
  output_file: flashcards.md

study_flashcards_task:
  description: >
    Go through the flashcards one by one with the user. The user can type "show" to d
isplay a card's front,
    "flip" to reveal its back, or "quit" to exit.
  expected_output: >
    Interactive session with the user. No final text is required, just user engagemen
t.
  agent: study_buddy
  context:
    - flashcard_task
  human_input: true
```

crew.py

Found in folder: **/src/buddy_ai**

This code defines a class `BuddyAi` that represents a crew in the CrewAI framework. The class defines several agents and tasks that work together to process a PDF document, including reading the document, summarizing its content, generating a quiz, and creating flashcards. The tasks are executed in a sequential process, and the crew is configured using YAML files.

```
import os
from crewai import Agent, Crew, Process, Task
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import PDFSearchTool

# If you want to run a snippet of code before or after the crew starts,
# you can use the @before_kickoff and @after_kickoff decorators
# https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators

PDF_PATH = 'C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf'

@CrewBase
class BuddyAi():
 """BuddyAi crew"""

 # Learn more about YAML configuration files here:
 # Agents: https://docs.crewai.com/concepts/agents#yaml-configuration-recommended
 # Tasks: https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended
 agents_config = 'config/agents.yaml'
 tasks_config = 'config/tasks.yaml'

 # If you would like to add tools to your agents, you can learn more about it here:
 # https://docs.crewai.com/concepts/agents#agent-tools
 @agent
 def pdf_reader(self) -> Agent:
  return Agent(
   config=self.agents_config['pdf_reader'],
   verbose=False,
   tools=[PDFSearchTool(pdf=PDF_PATH)]
  )

 @agent
 def summarizer(self) -> Agent:
  return Agent(
   config=self.agents_config['summarizer'],
   verbose=False
  )

 @agent
 def quiz_maker(self) -> Agent:
  return Agent(
   config=self.agents_config['quiz_maker'],
   verbose=False,
   strict_parser=False
  )
```

```
@agent
def study_buddy(self) -> Agent:
 return Agent(
  config=self.agents_config['study_buddy'],
  verbose=False,
 )

@agent
def flashcard_maker(self) -> Agent:
 return Agent(
  config=self.agents_config['flashcard_maker'],
  verbose=False,
  strict_parser=False,
  function_calling_llm=None
 )

# To learn more about structured task outputs,
# task dependencies, and task callbacks, check out the documentation:
# https://docs.crewai.com/concepts/tasks#overview-of-a-task
@task
def reading_task(self) -> Task:
 """
  This task will:
  - verify file existence and access
  - attempt multiple queries to extract full content
  - Return extracted content or clear out all error messages
 """
 def reading_logic(agent, inputs):
  # 0. verify the file path and acces
  if not os.path.isfile(PDF_PATH):
   return "ERROR: PDF file not found or not accessible at the given path."

  # 1. Attempt to retrieve content from the PDF using different queries.
  queries = ["", "all content", "document", "*"]
  extracted_content = None
  fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files."

  for q in queries:
   agent.think(f"Trying query '{q}' to get PDF content. ")
   result = agent.use_tool("Search a PDF's content", {"query": q})
   if result and fallback_snippet not in result:
    # trying to get something different from the fallback description
    extracted_content = result.strip()
    break

  # 2. confirm that content was extracted
  if not extracted_content:
   return (
    "ERROR: unable to extract meaningful text from the PDF."
    "Make sure that PDF has searchable text or try another document."
   )

  return extracted_content

 return Task(
```

```
   config=self.tasks_config['reading_task'],
   logic=reading_logic
  )

 @task
 def summarizing_task(self) -> Task:
  """
   Summarize the extracted PDF text.
    If the reading_task returned an ERROR,
     handle that gracefully here!
  """
  def summarizing_logic(agent, inputs, context):
   pdf_text = context.get('reading_task', '')
   if pdf_text.startswith("ERROR:"):
    return f"Cannot summarize because: {pdf_text}"

   return agent.llm(
    f"Summarize the following text into a concise overview:\n\n{pdf_text}"
   ).strip

  return Task(
   config=self.tasks_config['summarizing_task'],
   logic=summarizing_logic,
   output_file='output/summary.md'
  )

 @task
 def quiz_task(self) -> Task:
  return Task(
   config=self.tasks_config['quiz_task'],
  )

 @task
 def user_test_task(self) -> Task:
  def test_logic(agent, inputs, context):
   quiz_text = context.get('quiz_task', '')

   # the agent will prompt the user:
   agent.think("I will now ask the user to answer the quiz.")
   agent.say("Here are your quiz questions:\n")
   agent.say(quiz_text)
   agent.say("Please provide your answers in the format: A,B,C,... for each question
in order.")

   user_answers = inputs.get('human_input')

   user_answers_list = [ans.strip().upper() for ans in user_answers.split(",")]

   # Q1, A, b, c, Correct answer: A

   import re
   correct_answers = re.findall(r'Correct Answer:\s*([A-
D])', quiz_text, re.IGNORECASE)

   score = 0
```

```
   for user_ans, correct_ans in zip(user_answers_list, correct_answers):
    if user_ans == correct_ans.upper():
     score += 1

   total = len(correct_ans)
   result = f"You answered {score} out of {total} correctly!"

   agent.say(result)
   return result


  return Task(
   config=self.tasks_config['user_test_task'],
   logic=test_logic
  )

 @task
 def flashcard_task(self) -> Task:
  return Task(
   config=self.tasks_config['flashcard_task']
  )

 @task
 def study_flashcards_task(self) -> Task:
  def study_logic(agent, inputs, context):
   # parse the flashcards from the flashcard_task output
   # let the user type show, flip or quit to navigate them.
   flashcards_text = context.get('flashcards_task', '')
   if not flashcards_text.strip():
    return "No flashcards found. Possibly an error."

   agent.think("Parsing flashcards from text...")

   # 1) front: ...
   #    back: ...
   lines = [l.strip() for l in flashcards_text.split('\n') if l.strip()]

   flashcards = []
   current_fc = {}

   for line in lines:
    if line.lower().startswith("front:"):
     current_fc = {"front": line[6:].strip(), "back": ""}
    elif line.lower().startswith("back:"):
     current_fc["back"] = line[5:].strip()
     flashcards.append(current_fc)
     current_fc = {}

   if not flashcards:
    return "Couldn't parse any flashcards from the text."

   agent.say(f"I have {len(flashcards)} flashcards loaded. We'll go through them now.
\n")
   agent.say("Type 'show' to see the next flashcard, 'flip' to see its back, or 'quit
' to end.\n")
```

```
    index = 0
    showing_front = False
    user_input = inputs.get('human_input', '').lower()

    if user_input == "quit":
     return "Exiting flashcard study session."

    if index >= len(flashcards):
     return "No more flashcards left."

    if user_input == "show":
     agent.say(f"Flashcard {index + 1} front: {flashcards[index]['front']}")
     showing_front = True
     return "Type 'flip' next time to see the back, or 'quit' to stop."
    elif user_input == "flip" and not showing_front:
     return "You need to 'show' the card first. Then you can 'flip' it."
    elif user_input == "flip" and showing_front:
     # show the back
     agent.say(f"Flashcard {index+1} back: {flashcards[index]['back']}")
     return f"Now you can type 'show' to move to the next card or 'quit' to stop."
    else:
     return "Invalid command. Please type 'show' or 'flip' or 'quit'"


   return Task(
    config=self.tasks_config['study_flashcards_task'],
    logic=study_logic,
    human_input=True
   )

  @crew
  def crew(self) -> Crew:
   """Creates the BuddyAi crew"""
   # To learn how to add knowledge sources to your crew, check out the documentation:
   # https://docs.crewai.com/concepts/knowledge#what-is-knowledge

   return Crew(
    agents=self.agents, # Automatically created by the @agent decorator
    tasks=self.tasks, # Automatically created by the @task decorator
    process=Process.sequential,
    verbose=True,
    # process=Process.hierarchical, # In case you wanna use that instead https://docs.
crewai.com/how-to/Hierarchical/
   )
```

main.py

Found in folder: **/src/buddy_ai**

This code defines four functions: run, train, replay, and test, which interact with the BuddyAi crew to perform different actions. The functions take varying inputs, such as the number of iterations or a task ID, and execute the corresponding crew action. Each function also includes error handling to

catch and raise exceptions if any issues occur during execution.

```
#!/usr/bin/env python
import sys
import warnings

from buddy_ai.crew import BuddyAi

warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")

def run():
    """
    Run the crew.
    """
    # No inputs needed since tasks are self-contained
    inputs = {}
    BuddyAi().crew().kickoff(inputs=inputs)


def train():
    """
    Train the crew for a given number of iterations.
    """
    inputs = {}
    try:
        BuddyAi().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2], i
nputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")

def replay():
    """
    Replay the crew execution from a specific task.
    """
    try:
        BuddyAi().crew().replay(task_id=sys.argv[1])

    except Exception as e:
        raise Exception(f"An error occurred while replaying the crew: {e}")

def test():
    """
    Test the crew execution and returns the results.
    """
    inputs = {}
    try:
        BuddyAi().crew().test(n_iterations=int(sys.argv[1]), openai_model_name=sys.ar
gv[2], inputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while replaying the crew: {e}")
```


Complete Project with Local LLM

agents.yaml

Found in folder: **/src/buddy_ai/config**

This code defines five roles for a project, each with a specific goal and backstory. The roles include a PDF content extractor, AI content summarizer, quiz creator, quiz master, and flashcard generator. Each role is configured to use the Llama 3.2 model and has various settings such as verbosity and function calling.

```
pdf_reader:
  role: >
    PDF Content Extractor
  goal: >
    Extract the entire text content from a fixed PDF document and provide it.
    Use Llama 3.2 model.
  backstory: >
    You are skilled at retrieving text from PDF documents.
    The PDF is located at C:\Users\user\Downloads\Documents\sample_document.pdf.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

summarizer:
  role: >
    AI Content Summarizer
  goal: >
    Summarize the given text into a concise and informative overview.
  backstory: >
    You specialize in taking long pieces of text and producing a clear, concise summa
ry.
    Your summaries highlight the key points while maintaining factual accuracy.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

quiz_maker:
  role: >
    Quiz Creator
  goal: >
    Create a quiz (e.g., multiple-choice questions) based on given summarized text.
    Do not show the answers of the questions to the user.
  backstory: >
    Expert at formulating questions that test understanding of text content.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

study_buddy:
  role: >
    Quiz Master
  goal: >
    Interactively test the user based on the quiz generated.
  backstory: >
```

```
    You are a helpful study buddy that asks questions to the user and checks their an
swer.
  llm: llama-3.2
  verbose: false
  function_calling_llm: null

flashcard_maker:
  role: >
    Flashcard Generator
  goal: >
    Create flashcards from the summarized text to help the user study.
  backstory: >
    You turn summarized text into helpful question-
answer pairs, each representing a flashcard.
  llm: llama-3.2
  verbose: false
  strict_parser: false
  function_calling_llm: null
```

tasks.yaml

Found in folder: **/src/buddy_ai/config**

This code defines a series of tasks to process a PDF document, including extracting text, summarizing content, generating a quiz, creating flashcards, and interacting with the user to test their knowledge. Each task is described with its expected output and the agent responsible for executing it. The tasks are linked together through context dependencies, allowing them to build on each other's outputs.

```
reading_task:
  description: >
    Extract the entire text from the PDF document at C:\Users\user\Downloads\Document
s\sample_document.pdf.
    Return the raw text content without modification.
  expected_output: >
    The complete extracted text of the PDF document.
  agent: pdf_reader

summarizing_task:
  description: >
    Take the provided PDF text and summarize it into a concise overview that highligh
ts the main points.
  expected_output: >
    A concise summary (around 2-4 paragraphs) highlighting the key themes and facts f
rom the document.
  agent: summarizer
  context:
    - reading_task
  output_file: summary.md

quiz_task:
  description: >
    Using the summary of the PDF's content, create a quiz with multiple-
```



```
choice questions that test understanding of the main concepts.
    Provide say, 5 multiple-choice questions. Each question should have 4 options and
 indicate the correct answers.
  expected_output: >
    A quiz with 5 multiple-choice questions and their correct answers.
  agent: quiz_maker
  context:
    - summarizing_task
  output_file: quiz.md

user_test_task:
  description: >
    Present the quiz questions to the user and ask them to provide their answers.
    After the user responds, check the correctness and provide a final score.
  expected_output: >
    A summary of user performance, e.g. "You answered X out of Y questions correctly.
"
  agent: study_buddy
  context:
    - quiz_task
  human_input: true

flashcard_task:
  description: >
    Using the summary of the PDF's content, create 5 flashcards. Each card has a "fro
nt"
    (question or term) and a "back" (explanation).
  expected_output: >
    A structured list or text specifying each flashcard's front and back.
  agent: flashcard_maker
  context:
    - summarizing_task
  output_file: flashcards.md

study_flashcards_task:
  description: >
    Go through the flashcards one by one with the user. The user can type "show" to d
isplay a card's front,
    "flip" to reveal its back, or "quit" to exit.
  expected_output: >
    Interactive session with the user. No final text is required, just user engagemen
t.
  agent: study_buddy
  context:
    - flashcard_task
  human_input: true
```

crew.py

Found in folder: **/src/buddy_ai**

This code defines a class `BuddyAi` that represents a crew in the CrewAI framework. The class defines several agents and tasks that work together to process a PDF document, including reading

the document, summarizing its content, generating quiz questions, and creating flashcards. The tasks are executed in a sequential process, and the crew is configured to use specific language models and tools.

```
import os
from crewai import Agent, Crew, Process, Task, LLM
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import PDFSearchTool

# If you want to run a snippet of code before or after the crew starts,
# you can use the @before_kickoff and @after_kickoff decorators
# https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators

PDF_PATH = 'C:\\Users\\user\\Downloads\\Documents\\sample_document.pdf'

@CrewBase
class BuddyAi():
 """BuddyAi crew"""

 # Learn more about YAML configuration files here:
 # Agents: https://docs.crewai.com/concepts/agents#yaml-configuration-recommended
 # Tasks: https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended
 agents_config = 'config/agents.yaml'
 tasks_config = 'config/tasks.yaml'

 # If you would like to add tools to your agents, you can learn more about it here:
 # https://docs.crewai.com/concepts/agents#agent-tools
 @agent
 def pdf_reader(self) -> Agent:
  return Agent(
   config=self.agents_config['pdf_reader'],
   llm=LLM(
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
   ),
   verbose=False,
   tools=[PDFSearchTool(pdf=PDF_PATH)]
  )

 @agent
 def summarizer(self) -> Agent:
  return Agent(
   config=self.agents_config['summarizer'],
   llm=LLM(
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
   ),
   verbose=False
  )

 @agent
 def quiz_maker(self) -> Agent:
  return Agent(
   config=self.agents_config['quiz_maker'],
   llm=LLM(
```

```
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
   ),
   verbose=False,
   strict_parser=False
  )

@agent
def study_buddy(self) -> Agent:
  return Agent(
   config=self.agents_config['study_buddy'],
   llm=LLM(
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
   ),
   verbose=False,
  )

@agent
def flashcard_maker(self) -> Agent:
  return Agent(
   config=self.agents_config['flashcard_maker'],
   llm=LLM(
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
   ),
   verbose=False,
   strict_parser=False,
   function_calling_llm=None
  )

# To learn more about structured task outputs,
# task dependencies, and task callbacks, check out the documentation:
# https://docs.crewai.com/concepts/tasks#overview-of-a-task
@task
def reading_task(self) -> Task:
  """
   This task will:
   - verify file existence and access
   - attempt multiple queries to extract full content
   - Return extracted content or clear out all error messages
  """
  def reading_logic(agent, inputs):
   # 0. verify the file path and acces
   if not os.path.isfile(PDF_PATH):
    return "ERROR: PDF file not found or not accessible at the given path."

   # 1. Attempt to retrieve content from the PDF using different queries.
   # queries = ["", "all content", "document", "*"]
   extracted_content = None
   fallback_snippet = "PDF RAG the PDFSearchTool is designed to search PDF files."

   # for q in queries:
   agent.think(f"Trying query (all content) to get PDF content. ")
   result = agent.use_tool("Search a PDF's content", {"query": "all content"})
```

```
  if result and fallback_snippet not in result:
   # trying to get something different from the fallback description
   extracted_content = result.strip()

  # 2. confirm that content was extracted
  if not extracted_content:
   return (
    "ERROR: unable to extract meaningful text from the PDF."
    "Make sure that PDF has searchable text or try another document."
   )

  return extracted_content

 return Task(
  config=self.tasks_config['reading_task'],
  logic=reading_logic
 )

@task
def summarizing_task(self) -> Task:
 """
  Summarize the extracted PDF text.
   If the reading_task returned an ERROR,
    handle that gracefully here!
 """
 def summarizing_logic(agent, inputs, context):
  pdf_text = context.get('reading_task', '')
  if pdf_text.startswith("ERROR:"):
   return f"Cannot summarize because: {pdf_text}"

  return agent.llm(
   f"Summarize the following text into a concise overview:\n\n{pdf_text}"
  ).strip

 return Task(
  config=self.tasks_config['summarizing_task'],
  logic=summarizing_logic,
  output_file='output/summary.md'
 )

@task
def quiz_task(self) -> Task:
 return Task(
  config=self.tasks_config['quiz_task'],
 )

@task
def user_test_task(self) -> Task:
 def test_logic(agent, inputs, context):
  quiz_text = context.get('quiz_task', '')

  # the agent will prompt the user:
  agent.think("I will now ask the user to answer the quiz.")
  agent.say("Here are your quiz questions:\n")
  agent.say(quiz_text)
```

```
   agent.say("Please provide your answers in the format: A,B,C,... for each question
in order.")

   user_answers = inputs.get('human_input')

   user_answers_list = [ans.strip().upper() for ans in user_answers.split(",")]

   # Q1, A, b, c, Correct answer: A

   import re
   correct_answers = re.findall(r'Correct Answer:\s*([A-
D])', quiz_text, re.IGNORECASE)

   score = 0
   for user_ans, correct_ans in zip(user_answers_list, correct_answers):
    if user_ans == correct_ans.upper():
     score += 1

   total = len(correct_ans)
   result = f"You answered {score} out of {total} correctly!"

   agent.say(result)
   return result


  return Task(
   config=self.tasks_config['user_test_task'],
   logic=test_logic
  )

 @task
 def flashcard_task(self) -> Task:
  return Task(
   config=self.tasks_config['flashcard_task']
  )

 @task
 def study_flashcards_task(self) -> Task:
  def study_logic(agent, inputs, context):
   # parse the flashcards from the flashcard_task output
   # let the user type show, flip or quit to navigate them.
   flashcards_text = context.get('flashcards_task', '')
   if not flashcards_text.strip():
    return "No flashcards found. Possibly an error."

   agent.think("Parsing flashcards from text...")

   # 1) front: ...
   #    back: ...
   lines = [l.strip() for l in flashcards_text.split('\n') if l.strip()]

   flashcards = []
   current_fc = {}

   for line in lines:
```

```
    if line.lower().startswith("front:"):
     current_fc = {"front": line[6:].strip(), "back": ""}
    elif line.lower().startswith("back:"):
     current_fc["back"] = line[5:].strip()
     flashcards.append(current_fc)
     current_fc = {}

   if not flashcards:
    return "Couldn't parse any flashcards from the text."

   agent.say(f"I have {len(flashcards)} flashcards loaded. We'll go through them now.
\n")
   agent.say("Type 'show' to see the next flashcard, 'flip' to see its back, or 'quit
' to end.\n")

   index = 0
   showing_front = False
   user_input = inputs.get('human_input', '').lower()

   if user_input == "quit":
    return "Exiting flashcard study session."

   if index >= len(flashcards):
    return "No more flashcards left."

   if user_input == "show":
    agent.say(f"Flashcard {index + 1} front: {flashcards[index]['front']}")
    showing_front = True
    return "Type 'flip' next time to see the back, or 'quit' to stop."
   elif user_input == "flip" and not showing_front:
    return "You need to 'show' the card first. Then you can 'flip' it."
   elif user_input == "flip" and showing_front:
    # show the back
    agent.say(f"Flashcard {index+1} back: {flashcards[index]['back']}")
    return f"Now you can type 'show' to move to the next card or 'quit' to stop."
   else:
    return "Invalid command. Please type 'show' or 'flip' or 'quit'"


  return Task(
   config=self.tasks_config['study_flashcards_task'],
   logic=study_logic,
   human_input=True
  )

 @crew
 def crew(self) -> Crew:
  """Creates the BuddyAi crew"""
  # To learn how to add knowledge sources to your crew, check out the documentation:
  # https://docs.crewai.com/concepts/knowledge#what-is-knowledge

  return Crew(
   agents=self.agents, # Automatically created by the @agent decorator
   tasks=self.tasks, # Automatically created by the @task decorator
   process=Process.sequential,
```

```
   verbose=True,
   # process=Process.hierarchical, # In case you wanna use that instead https://docs.
crewai.com/how-to/Hierarchical/
  )
```

main.py

Found in folder: **/src/buddy_ai**

This code defines four functions: run, train, replay, and test, which interact with the BuddyAi crew to perform different actions. The functions take varying inputs, such as the number of iterations or a task ID, and execute the corresponding crew action. Each function also includes error handling to catch and raise exceptions if any issues occur during execution.

```
#!/usr/bin/env python
import sys
import warnings

from buddy_ai.crew import BuddyAi

warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")

def run():
    """
    Run the crew.
    """
    # No inputs needed since tasks are self-contained
    inputs = {}
    BuddyAi().crew().kickoff(inputs=inputs)


def train():
    """
    Train the crew for a given number of iterations.
    """
    inputs = {}
    try:
        BuddyAi().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2], i
nputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")

def replay():
    """
    Replay the crew execution from a specific task.
    """
    try:
        BuddyAi().crew().replay(task_id=sys.argv[1])

    except Exception as e:
        raise Exception(f"An error occurred while replaying the crew: {e}")
```

```
def test():
    """
    Test the crew execution and returns the results.
    """
    inputs = {}
    try:
        BuddyAi().crew().test(n_iterations=int(sys.argv[1]), openai_model_name=sys.ar
gv[2], inputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while replaying the crew: {e}")
```